시스템 프로그래밍 팀과제 #1 — 이진 파일 다루기 (hw2BMP)

조원 명단 및 역할

이름	학번	역할
전도원	22300650	과제 1-1 (헤더 출력), 1-2 (body 16진수 출력) 수행
송준선	22300404	과제 1-3 (grayscale 변환) 수행, 데모 캡쳐

제출 범위 및 검증 결과

이번 제출 범위는 과제 #1의 1-1, 1-2, 1-3이다. 각각 BMP 헤더 출력, 본문의 16진수 출력, 회색조 변환을 구현했다. 세 기능은 파일별 함수로 나누고, 조건문과 반복문을 중심으로 구현했다. 조원별 역할은 기존 기록을 유지했다.

본문 출력에서는 입력 파일과 출력 파일이 같을 때 원본이 손상되지 않도록 검사를 추가했다. same_file 함수는 파일의 장치 번호와 아이노드 번호를 비교하므로, 경로가 같거나 하드 링크·심볼릭 링크로 같은 파일을 가리키는 경우에도 출력 파일을 열기 전에 오류로 처리한다.

make로 컴파일하고 make test로 자동 검사를 실행한다. 최종 보고서는

report.pdf이며, 이 문서는 수정 가능한 보고서 원본이다. macOS의 GCC 명령 (실제 컴파일러는 Apple Clang)으로 빌드하고 기존 검사 20개가 모두 통과한 것을 확인했다. 헤더 값, 본문의 전체 바이트와 상대주소, 회색조 픽셀 값을 원본에서 직접 읽어 계산한 값과 비교했다. 검증에 사용한 khj.bmp는 비압축 32비트 BMP이다. 회색조 변환 코드는 24비트와 32비트 이미지를 처리하며, 그 밖의 BMP 인코딩은 이번 검증 범위에 포함하지 않았다.

첨부한 시연 이미지는 기존 세 기능의 실제 화면 캡처이다. 다시 실행한 검사의 전체 출력은 test-results.txt에 포함했다. 입력·출력을 같은 파일로 지정한 경우에도 오류로 종료하고 원본 바이트를 보존하는지 별도로 확인했다.

코드를 이해하려면 먼저 bmp.h에서 파일 헤더 구조체와 함수 선언을 읽고,

main.c에서 옵션에 따른 함수 호출을 확인한다. bmp.c는 공통 헤더 읽기와 파일 동일성 검사를 담당하고, header.c는 헤더 필드를 출력한다.

hexdump.c는 bfOffBits 위치로 이동하여 본문을 8바이트씩 묶어 출력한다.

grayscale.c는 패딩을 포함한 행 위치를 계산하고 B·G·R에 같은 밝기를 쓴다.

test.sh는 od, awk, cmp 등의 도구로 이 동작을 검사한다.

AI 도움 내역: 이번 제출 준비에서 OpenAI Codex의 도움을 받아 요구사항과 기존 코드를 검토하고, 동일 파일 출력 방지 기능을 추가하고, 검증을 실행했다. 또한 파일 비교 조건을 단계별로 풀고 회색조 변환 변수 이름을 정리했다. 파일 크기는 file_size, 픽셀당 바이트 수는 bytes_per_pixel로 표시했다. 제출 범위와 코드 설명을 보완하고 보고서를 한글 PDF로 변환했으며, ZIP 파일 구성과 압축 해제 후 재검증을 수행했다. 조원 역할 기록과 원래 시연 화면은 유지했다.

1. 과제 개요

BMP 이미지 파일을 이진 모드로 읽어서 다음 세 가지 기능을 옵션으로 실행하는 프로그램을 만들었다.

과제	명령	기능
1-1	hw2BMP -h <input_bmp_file>	BMP 헤더 내용을 화면에 출력

1-2	hw2BMP -o <input_bmp_file> <out_file>	BMP body를 16진수 텍스트로 출력 (한 줄 8 바이트, 앞에 상대주소)
1-3	hw2BMP -g <input_bmp_file> <output_bmp_file>	grayscale로 바꿔서 새 BMP 파일로 저장

2. 파일 구성

파일	설명
Makefile	컴파일 규칙 (make, make test, make clean)
bmp.h	BMP 헤더 구조체와 함수 선언
bmp.c	공통 헤더 읽기 및 입력·출력 파일 동일성 검사
header.c	과제 1-1 헤더 출력
hexdump.c	과제 1-2 body 16진수 출력
grayscale.c	과제 1-3 grayscale 변환
main.c	옵션을 확인하고 기능 실행
test.sh	khj.bmp 로 세 기능을 자동 테스트하는 스크립트

3. 자료구조 설명

BMP 파일은 다음 순서로 되어 있다.

```
0바이트     : BITMAPFILEHEADER (14바이트)
14바이트    : BITMAPINFOHEADER (40바이트)
bfOffBits   : 픽셀 데이터 (body)
```

- 픽셀은 B, G, R (32비트는 B, G, R, A) 순서로 저장된다.

- 한 줄(row)의 크기는 4의 배수가 되도록 뒤에 0이 채워진다 (패딩).

- 보통 이미지의 아래쪽 줄부터 저장된다.

BITMAPFILEHEADER (14바이트)

멤버	타입	의미
bfType	unsigned short	파일 종류, 'BM'
bfSize	unsigned int	파일 전체 크기
bfReserved1, bfReserved2	unsigned short	사용 안 함 (0)
bfOffBits	unsigned int	픽셀 데이터가 시작하는 위치

BITMAPINFOHEADER (40바이트)

멤버	타입	의미
biSize	unsigned int	정보 헤더 크기 (40)
biWidth, biHeight	int	가로, 세로 픽셀 수
biPlanes	unsigned short	항상 1
biBitCount	unsigned short	픽셀 하나의 비트 수 (24, 32 등)
biCompression	unsigned int	압축 방식 (0 = 압축 안 함)
biSizeImage	unsigned int	픽셀 데이터 크기 (0일 수 있음)
biXPelsPerMeter, biYPelsPerMeter	int	해상도
biClrUsed, biClrImportant	unsigned int	색상 수

#pragma pack(1) 을 쓴 이유

- C 컴파일러는 구조체 멤버를 4바이트 단위로 맞추려고 사이에 빈 공간을 넣는다.

- 그러면 bfType(2바이트) 뒤에 2바이트가 비어서 BITMAPFILEHEADER가 14바이트가 아니라 16바이트가 된다.

- 이 상태로 fread로 읽으면 값이 밀린다.

- 그래서 #pragma pack(1)로 빈 공간 없이 1바이트 단위로 붙이도록 했다.

4. 함수 설명

bmp.c

함수	기능
int read_header(FILE *fp, BITMAPFILEHEADER *fh, BITMAPINFOHEADER *ih)	fread로 두 헤더를 읽고, bfType이 'BM'(0x4D42)인지 확인한다. 성공하면 0, 실패하면 -1
int same_file(const char *first, const char *second)	두 경로의 장치 번호와 아이노드 번호를 비교하여 같은 파일이면 참을 반환한다. 하드 링크와 심볼릭 링크도 검사한다

header.c (과제 1-1)

함수	기능
int print_header(char *filename)	파일을 "rb"로 열고 read_header로 헤더를 읽어서, 모든 멤버 값을 출력한다

hexdump.c (과제 1-2)

함수	기능
int hex_dump(char *in_name, char *out_name)	헤더를 읽은 뒤 fseek으로 bfOffBits 위치로 이동한다. 파일 끝까지 fgetc로 한 바이트씩 읽어 %02X로 출력한다. 읽은 바이트 수 count를 상대주소로 쓰고, count % 8 == 0일 때마다 줄 앞에 주소를 %08X로 붙인다

grayscale.c (과제 1-3)

함수	기능
int make_gray(char *in_name, char *out_name)	파일 전체를 malloc한 메모리에 읽는다. 행/열 이중 for문으로 픽셀마다 밝기를 계산해 B, G, R에 같은 값을 넣고, 결과를 "wb"로 저장한다. 24비트와 32비트 BMP를 지원한다

grayscale 계산 방법

- 밝기 = 0.299 R + 0.587 G + 0.114 B 공식을 쓴다.

- 실수 계산 대신 (299*R + 587*G + 114*B + 500) / 1000 처럼 정수로 계산한다. +500은 반올림을 위한 것이다.

- 한 줄의 크기는 (width * bytes_per_pixel + 3) / 4 * 4 로 구해서 패딩을 건너뛴다.

- 32비트의 4번째 바이트(알파)는 바꾸지 않는다.

main.c

함수	기능
void print_usage(char *name)	사용법 출력
int main(int argc, char *argv[])	argv[1] 옵션과 인자 개수를 확인해서 알맞은 함수를 실행한다. 잘못 입력하면 사용법을 출력하고 1을 반환한다

5. 프로그램 사용법

```
# 컴파일
$ make

# 1-1) 헤더 출력
$ ./hw2BMP -h khj.bmp

# 1-2) body를 16진수 텍스트로 저장
$ ./hw2BMP -o khj.bmp body.txt

# 1-3) grayscale 변환
$ ./hw2BMP -g khj.bmp khj_gray.bmp

# 자동 테스트
$ make test

# 정리
$ make clean
```

6. 실행 결과 (데모)

컴파일 — make

빌드

make — 빌드

```
$ make
gcc -Wall -c main.c
gcc -Wall -c bmp.c
gcc -Wall -c header.c
gcc -Wall -c hexdump.c
gcc -Wall -c grayscale.c
gcc -Wall -o hw2BMP main.o bmp.o header.o hexdump.o grayscale.o
```

1-1) ./hw2BMP -h khj.bmp

khj.bmp는 71×65 픽셀의 32비트 이미지이고, 압축은 하지 않았다. 픽셀 데이터는 파일의 54번째 바이트부터 시작한다.

1-1 헤더 출력

1-1) hw2BMP -h

```
$ ./hw2BMP -h khj.bmp
===== BITMAP FILE HEADER =====
bfType          : BM
bfSize          : 18514
bfReserved1     : 0
bfReserved2     : 0
bfOffBits       : 54

===== BITMAP INFO HEADER =====
biSize          : 40
biWidth         : 71
biHeight        : 65
biPlanes        : 1
biBitCount      : 32
biCompression   : 0
biSizeImage     : 0
biXPelsPerMeter : 0
biYPelsPerMeter : 0
biClrUsed       : 0
biClrImportant  : 0
```

1-2) ./hw2BMP -o khj.bmp body.txt

- body 크기는 71 × 65 × 4 = 18460 바이트이고, 한 줄에 8바이트씩이라 2308줄이 나온다.

- 마지막 줄 주소 0x4818 = 18456 이므로 상대주소가 맞게 증가한다.

1-2 16진수 출력

1-2) hw2BMP -o

```
$ ./hw2BMP -o khj.bmp body.txt
BMP body 18460 바이트를 body.txt 에 저장했습니다.
$ head -n 8 body.txt
00000000: 00 01 02 FF 00 00 01 FF
00000008: 00 00 00 FF 03 03 03 FF
00000010: 03 01 01 FF 04 02 02 FF
00000018: C1 BF BF FF 96 94 94 FF
00000020: 04 00 01 FF 52 4F 51 FF
00000028: DE DD E1 FF 3E 3E 44 FF
00000030: 88 8B 90 FF 19 1C 24 FF
00000038: 00 01 09 FF 14 1C 29 FF
...
$ tail -n 3 body.txt
00004808: A1 99 9A FF A2 9A 9B FF
00004810: A2 9A 9B FF A3 9B 9C FF
00004818: A3 9B 9C FF
$ wc -l body.txt
    2308 body.txt
```

1-3) ./hw2BMP -g khj.bmp khj_gray.bmp

1-3 grayscale 실행

1-3) hw2BMP -g

```
$ ./hw2BMP -g khj.bmp khj_gray.bmp
grayscale 이미지를 khj_gray.bmp 에 저장했습니다.
$ file khj_gray.bmp
khj_gray.bmp: PC bitmap, Windows 3.x format, 71 x 65 x 32, cbSize 18514, bits offset 54
```

원본과 변환 결과 비교 (보기 쉽게 5배 확대):

1-3 원본/변환 비교

원본 khj.bmp



변환 khj_gray.bmp



잘못된 사용

인자가 맞지 않으면 사용법을 출력하고, 없는 파일이나 BMP가 아닌 파일은 오류 메시지를 출력한다.

오류 처리

잘못된 사용 처리

```
$ ./hw2BMP
사용법:
  ./hw2BMP -h <input_bmp_file>
  ./hw2BMP -o <input_bmp_file> <out_file>
  ./hw2BMP -g <input_bmp_file> <output_bmp_file>
$ ./hw2BMP -h nofile.bmp
파일을 열 수 없습니다: nofile.bmp
$ ./hw2BMP -h Makefile
BMP 파일이 아닙니다: Makefile
```

7. 테스트

make test 를 실행하면 test.sh 가 khj.bmp 로 세 기능을 검사한다. 정답은 od 명령으로 khj.bmp 를 직접 읽어서 만들고, 프로그램 결과와 비교한다.

대상	검사 내용
1-1 -h	bfType, bfSize, bfOffBits, biWidth, biHeight, biBitCount 가 실제 파일 값과 같은지

1-2 -o	줄 수, 첫 줄 주소, 전체 내용이 정답과 같은지
1-3 -g	파일 크기와 헤더가 원본과 같은지, 모든 픽셀이 밝기 공식대로 바뀌었는지
잘못된 사용	인자 없음, 없는 옵션, 인자 부족, 없는 파일, BMP 아닌 파일일 때 실패하는지

테스트 결과

make test

```
$ make test
sh test.sh
테스트 파일: khj.bmp (71x65, 32비트, body 18460 바이트)

== 1-1) -h 헤더 출력 ==
  [PASS] 실행 성공
  [PASS] bfType = BM
  [PASS] bfSize = 18514
  [PASS] bfOffBits = 54
  [PASS] biWidth = 71
  [PASS] biHeight = 65
  [PASS] biBitCount = 32

== 1-2) -o 16진수 출력 ==
  [PASS] 실행 성공
  [PASS] 줄 수 = 2308
  [PASS] 첫 줄 주소가 00000000
  [PASS] 전체 내용이 정답과 같음

== 1-3) -g grayscale 변환 ==
  [PASS] 실행 성공
  [PASS] 파일 크기가 원본과 같음
  [PASS] 헤더가 원본과 같음
  [PASS] 모든 픽셀이 밝기 공식대로 흑백 변환됨

== 잘못된 사용 ==
  [PASS] 인자 없음 -> 실패
  [PASS] 없는 옵션 -> 실패
  [PASS] 인자 부족 -> 실패
  [PASS] 없는 파일 -> 실패
  [PASS] BMP 아닌 파일 -> 실패

결과: PASS 20 / FAIL 0
```